

Esercizio 4 - Stazioni di rifornimento

Indice

Indice.....	1
Introduzione.....	2
Soluzione A.....	2
Metodologia.....	2
Analisi dei costi.....	3
Soluzione B.....	3
Metodologia.....	3
Analisi dei costi.....	4
Formato di input e output con esempi.....	4

Introduzione

Si ha una strada lunga L con N stazioni di rifornimento in posizioni distinte, si ha la possibilità di chiudere al massimo M stazioni, escluse la stazione di inizio, quella di fine, e una stazione intermedia denominata “stazione di controllo”, al fine di massimizzare la spaziatura del percorso, ovvero la distanza minima tra due stazioni consecutive

Soluzione A

Metodologia

L'idea alla base di questa soluzione è quella di utilizzare una **ricerca lineare decrescente** basata sulla **spaziatura**. In questo caso per **valori di D decrescente** viene utilizzata la funzione `verify_path` che ritorna un valore **booleano *true*** se, fissato quel valore di D rappresentante la spaziatura, si riesce a creare un percorso, chiudendo al massimo M stazioni, che **ha spaziatura D** , ritornando *false* altrimenti. In questo modo quando si trova una soluzione per un certo valore di D **si ha la certezza** che quel valore sia la **spaziatura massima**.

Analisi dei costi

Per calcolare la **complessità computazionale** di questa idea basta analizzare il **comportamento** dei due cicli *for* presenti all'interno del *main* e della funzione `verify_path`: in particolare dentro il *main* si ha un ciclo *for* con **L iterazioni**, pertanto ha costo $O(L)$. All'interno del ciclo però viene **invocata** la funzione `verify_path` che ha a sua volta un altro ciclo *for* che **ripete N volte** e quindi ha costo $O(N)$. Quindi il costo totale è:

$$O(L \cdot N)$$

Per quanto riguarda invece la **complessità spaziale** questa è data principalmente dalla dimensione dell'array *points* di **dimensione $N + 2$** , questo perchè tutte le altre **variabili hanno costo spaziale costante**, quindi la **complessità spaziale** è:

$$O(N + 2) = O(N)$$

Complessità computazionale	$O(L \cdot N)$	<i>Dovuto ai due cicli for per la ricerca lineare</i>
Complessità spaziale	$O(N)$	<i>Dovuto all'array points</i>

Soluzione B

Metodologia

L'idea è quella di, invece di provare tutte le soluzioni a partire da L, effettuare una **ricerca binaria** ponendo come valore minimo iniziale *low* 1 e valore massimo *high* L. Ad ogni iterazione si ricava il **valore intermedio** *mid* sulla quale viene invocata la funzione *is_valid* per controllare se con quel valore di spaziatura **si ha una soluzione ammissibile**. Se quella soluzione è ammissibile significa che tutta la **parte sinistra**, quindi quella che va da *low* a *mid*, è ammissibile però sicuramente **non è ottima** e di conseguenza la ricerca si sposta sulla **parte destra dell'intervallo di valori**, in caso invece *mid* **non appartenga** all'insieme di soluzioni ammissibili allora significa che neanche i valori della **parte destra** dell'intervallo non saranno ammissibili a loro volta e quindi si dovranno esaminare solamente i valori della **parte sinistra**. Questo procedimento si ripete **iterativamente** fin quando **low non diventa maggiore di high**, dopo la quale si prende l'ultimo valore valido è *best_D* che coincide con la spaziatura massima e quindi la soluzione ottima.

Analisi dei costi

Essendo applicato il concetto di **ricerca binaria** il ciclo *while* ha costo $O(\log(L))$, al suo interno viene richiamata la funzione *is_valid* che itera su **tutte le stazioni** e che quindi ha costo $O(N)$, di conseguenza il **costo computazionale** totale è:

$$O(N \log(L))$$

Analizzando la **complessità spaziale** invece come nel caso precedente si ha sempre un array di dimensione N che memorizza le stazioni di rifornimento, quindi è così calcolato:

$$O(N)$$

Complessità computazionale	$O(N \log(L))$	<i>Dovuto alla ricerca binaria</i>
Complessità spaziale	$O(N)$	<i>Dovuto alla dimensione di stations</i>

Formato di input e output con esempi

Il formato di input e output è uguale per entrambe le soluzioni, con n = 5 si ha:

Input:

<L> <N> <M> <stazione 1> <stazione 2> ... <stazione N> <stazione obbligatoria C>
20 7 4 2 5 8 10 12 15 18 10

Output:

Spaziatura massima: 5

Stazioni da chiudere: 2 8 12 18